

Network Layer: IPv4 and IPv6 Packet Structures

Theory and Linux CLI Parsing

Lecturer

April 14, 2026

Presentation Outline

- 1 Part 1: IPv4 Packet Structure
- 2 Part 2: IPv6 Packet Structure
- 3 Memorization Schematics

Note: All theoretical concepts are mapped to Chapter 4 (The Network Layer: Data Plane) of Kurose and Ross, 8th Edition.

IPv4: Version & Header Length (Theory)

Book Reference: Kurose & Ross, Section 4.3.1

- **Bit Offsets:** Version (Bits 0-3), Header Length (Bits 4-7)
- **Typical Values:** Version = 4, Header Length = 5 (meaning 20 bytes).
- **Verbose Explanation:**
 - The **Version** field strictly specifies the IP protocol version. A router looks at this 4-bit sequence to determine how to interpret the remainder of the datagram.
 - The **Header Length** (IHL) is required because an IPv4 header can contain a variable number of options.
 - The value is measured in 32-bit words. A typical value of 5 means $5 \times 4 = 20$ bytes, which is the standard size of an IPv4 header without options.

IPv4: Version & Header Length (Linux CLI)

Parsing in Linux CLI:

```
$ sudo tcpdump -c 1 -v -xx -i eth0 ip
```

Verbose Explanation of Command:

- `-c 1`: Instructs tcpdump to capture exactly 1 packet and stop.
- `-v`: Verbose output. Translates the header fields into human-readable text.
- `-xx`: Prints the raw data of the packet in hexadecimal format.
- `-i eth0`: Specifies the interface to listen on.
- `ip`: The Berkeley Packet Filter (BPF) primitive to only catch IPv4 traffic.
- **What to look for:** In the hex output, the very first byte will be 45. The '4' represents the Version (IPv4), and the '5' represents the Header Length (5 words = 20 bytes).

IPv4: Type of Service / DiffServ (Theory)

Book Reference: Kurose & Ross, Section 4.3.1

- **Bit Offsets:** Bits 8-15
- **Typical Values:** 0x00 (Best Effort/Routine), 0xb8 (Expedited Forwarding).
- **Verbose Explanation:**
 - Originally called Type of Service (ToS), now repurposed as Differentiated Services (DiffServ).
 - This 8-bit field allows network administrators to prioritize certain types of traffic over others.
 - For example, real-time Voice over IP (VoIP) traffic requires low delay and jitter, so it receives a higher priority code than standard bulk HTTP or FTP data transfers.

Parsing in Linux CLI:

```
$ sudo tcpdump -v -n icmp
```

Verbose Explanation of Command:

- **-v:** Triggers verbose mode, which forces tcpdump to decode the ToS field.
- **-n:** Disables DNS resolution. It forces the CLI to display raw IP addresses instead of hostnames, speeding up the capture output significantly.
- **icmp:** Filters the capture to only show Ping requests/replies, which are easy to generate for testing.
- **What to look for:** The output will display a tag such as `tos 0x00`. This confirms the packet is being routed with standard "best-effort" priority.

IPv4: Datagram Length (Theory)

Book Reference: Kurose & Ross, Section 4.3.1

- **Bit Offsets:** Bits 16-31
- **Typical Values:** Ranges from 40 to 1500 bytes.
- **Verbose Explanation:**
 - This 16-bit field indicates the total length of the entire IP datagram, which includes both the 20-byte header and the encapsulated payload (data).
 - Because the field is 16 bits long, the theoretical maximum size of an IP datagram is $2^{16} - 1 = 65,535$ bytes.
 - However, datagrams rarely exceed 1,500 bytes to ensure they fit within a standard Ethernet frame's Maximum Transmission Unit (MTU) without needing fragmentation.

IPv4: Datagram Length (Linux CLI)

Parsing in Linux CLI:

```
$ ping -s 100 8.8.8.8 > /dev/null &  
$ sudo tcpdump -v -c 1 icmp
```

Verbose Explanation of Command:

- `ping -s 100`: Generates a background ICMP packet with exactly 100 bytes of payload.
- `-c 1`: Captures only the first packet of this flow.
- **What to look for:** In the tcpdump output, you will see `length 128`.
- **Why 128?:** 100 bytes (ping payload) + 8 bytes (ICMP header) + 20 bytes (IPv4 header) = 128 bytes total datagram length. The CLI perfectly verifies the math!

IPv4: Fragmentation Fields (Theory)

Book Reference: Kurose & Ross, Section 4.3.1

- **Bit Offsets:** Identifier (32-47), Flags (48-50), Offset (51-63)
- **Typical Values:** Flags = DF (Don't Fragment), Offset = 0.
- **Verbose Explanation:**
 - If an IP datagram exceeds the Maximum Transmission Unit (MTU) of a router's outgoing link, the router must slice it into smaller fragments.
 - **Identifier:** A 16-bit stamp given by the sender so the receiver knows which fragments belong to the same original packet.
 - **Flags:** Indicates if there are "More Fragments" (MF flag) coming, or instructs the router "Don't Fragment" (DF flag) this packet.
 - **Offset:** Tells the receiving host where this specific fragment belongs in the reassembled packet.

IPv4: Fragmentation Fields (Linux CLI)

Parsing in Linux CLI:

```
$ ping -s 3000 8.8.8.8 &  
$ sudo tcpdump -v -n icmp
```

Verbose Explanation of Command:

- **-s 3000:** We intentionally force the ping payload to 3,000 bytes. Since standard Ethernet MTU is 1,500 bytes, this forces the Linux kernel to fragment the packet before it even leaves the interface.
- **What to look for:** Tcpdump will output two separate IP packets sharing the exact same id.
- The first packet will show flags `[+]` (meaning the "More Fragments" bit is set to 1) and offset 0. The second packet will show flags `[none]` and an offset 1480, proving fragmentation occurred.

IPv4: Time-to-Live / TTL (Theory)

Book Reference: Kurose & Ross, Section 4.3.1

- **Bit Offsets:** Bits 64-71
- **Typical Values:** 64 (Linux/macOS), 128 (Windows), 255 (Cisco).
- **Verbose Explanation:**
 - The TTL field prevents packets from circulating endlessly in the network due to misconfigured routing tables (routing loops).
 - Despite the name "Time", it actually operates as a hop counter.
 - Every single router that processes the datagram must decrement this 8-bit field by exactly 1.
 - If the TTL field reaches 0, the router drops the packet and sends an ICMP "Time Exceeded" error message back to the source.

IPv4: Time-to-Live / TTL (Linux CLI)

Parsing in Linux CLI:

```
$ traceroute google.com  
$ sudo tcpdump -v -n host google.com
```

Verbose Explanation of Command:

- **traceroute:** This CLI utility weaponizes the TTL field. It intentionally sends packets with a TTL of 1, then 2, then 3, mapping every router along the path by capturing the resulting ICMP drop errors.
- **host google.com:** This BPF filter tells tcpdump to only capture traffic where Google is the source or destination.
- **What to look for:** In the tcpdump output, you will physically see the `ttl` value explicitly written, e.g., `ttl 64` changing to `ttl 63` as it crosses router boundaries.

IPv4: Upper-Layer Protocol (Theory)

Book Reference: Kurose & Ross, Section 4.3.1

- **Bit Offsets:** Bits 72-79
- **Typical Values:** 6 (TCP), 17 (UDP), 1 (ICMP).
- **Verbose Explanation:**
 - This 8-bit field is the critical "glue" that binds the Network layer to the Transport layer.
 - When an IP datagram reaches its final destination, the host needs to know which specific transport-layer protocol should process the payload.
 - If the value is 6, the payload is passed to the TCP socket software. If the value is 17, it goes to UDP. Without this field, demultiplexing would be impossible.

IPv4: Upper-Layer Protocol (Linux CLI)

Parsing in Linux CLI:

```
$ curl http://example.com &  
$ sudo tcpdump -v -n tcp port 80
```

Verbose Explanation of Command:

- `curl`: A command-line web browser. Generating HTTP traffic guarantees we are using the TCP transport protocol.
- `tcp port 80`: A strict filter ensuring we only capture web traffic.
- **What to look for:** The verbose `tcpdump` output translates the raw integer into text for you. You will see `proto TCP (6)`.
- If you capture a DNS query instead, it would read `proto UDP (17)`, proving the router successfully parsed the 72-79 bit offset.

IPv4: Source & Destination Addresses (Theory)

Book Reference: Kurose & Ross, Section 4.3.1

- **Bit Offsets:** Source (Bits 96-127), Destination (Bits 128-159)
- **Typical Values:** Dotted-decimal notation (e.g., 192.168.1.5).
- **Verbose Explanation:**
 - These two 32-bit fields represent the core essence of the Internet Protocol: endpoint identification.
 - The Source Address lets the receiving host know exactly where to send replies.
 - The Destination Address is the most heavily scrutinized field in the entire header; routers use this 32-bit integer to execute their Longest Prefix Match algorithms against their forwarding tables to determine the next hop.

IPv4: Source & Destination Addresses (Linux CLI)

Parsing in Linux CLI:

```
$ sudo tcpdump -n -nn
```

Verbose Explanation of Command:

- **-n**: Instructs tcpdump not to resolve IP addresses to hostnames (e.g., it will show 8.8.8.8 instead of dns.google.com). This is critical for reading the raw fields.
- **-nn**: An even stricter flag that prevents tcpdump from translating port numbers to names (e.g., it shows 443 instead of https).
- **What to look for**: The output will explicitly format the source and destination separated by a greater-than sign indicating flow direction:
- 192.168.1.25.54321 > 8.8.8.8.53. Notice how tcpdump conveniently appends the Transport layer port number to the IP layer address.

Book Reference: Kurose & Ross, Section 4.3.4

- **Bit Offsets:** Bits 0-3
- **Typical Values:** 6
- **Verbose Explanation:**
 - Just like IPv4, the very first 4 bits of an IPv6 datagram identify the IP version number.
 - For IPv6, this binary sequence is always 0110 (which is 6 in decimal).
 - Unlike IPv4, there is no "Header Length" field immediately following this. Why? Because the designers of IPv6 streamlined the protocol to use a strict, fixed-length 40-byte header, entirely eliminating the need to measure it dynamically.

IPv6: Version (Linux CLI)

Parsing in Linux CLI:

```
$ sudo tcpdump -c 1 -xx ip6
```

Verbose Explanation of Command:

- **ip6:** This is the BPF primitive that completely filters out legacy IPv4 traffic, capturing only native IPv6 traffic crossing the interface.
- **-xx:** Dumps the raw Link-layer and Network-layer bytes in purely hexadecimal format.
- **What to look for:** Bypassing the 14-byte Ethernet header in the hex output, the first byte of the IP payload will begin with the digit 6. This 6 directly correlates to the 4-bit binary 0110, verifying to the OS kernel that an IPv6 stack should parse the packet.

IPv6: Traffic Class (Theory)

Book Reference: Kurose & Ross, Section 4.3.4

- **Bit Offsets:** Bits 4-11
- **Typical Values:** 0x00 to 0xff.
- **Verbose Explanation:**
 - This 8-bit field is the direct IPv6 equivalent to the IPv4 Type of Service (ToS) / Differentiated Services field.
 - It serves the exact same purpose: allowing routers to categorize and prioritize datagrams based on Quality of Service (QoS) requirements.
 - During network congestion, routers will look at the Traffic Class bits to determine if a packet containing latency-sensitive video should jump ahead of a packet containing an email.

IPv6: Traffic Class (Linux CLI)

Parsing in Linux CLI:

```
$ sudo tshark -i eth0 -O ipv6 -c 1 -Y "ipv6"
```

Verbose Explanation of Command:

- **tshark:** The terminal-based version of Wireshark. We use this here because tshark parses deeply nested IPv6 fields much more cleanly than standard tcpdump.
- **-O ipv6:** Instructs tshark to deeply expand only the IPv6 protocol tree, printing every single bit and byte mapping to the terminal.
- **-Y "ipv6":** The display filter ensuring we only print IPv6 packets.
- **What to look for:** The output will explicitly list a line reading Traffic Class: 0x00 (DSCP: CS0, ECN: Not-ECT). Tshark breaks the 8 bits down into the 6-bit DSCP and 2-bit ECN components.

IPv6: Flow Label (Theory)

Book Reference: Kurose & Ross, Section 4.3.4

- **Bit Offsets:** Bits 12-31
- **Typical Values:** A 20-bit random hex value (e.g., 0x1a2b3), or 0 if unused.
- **Verbose Explanation:**
 - This is a brand new 20-bit field that has no equivalent in IPv4.
 - It allows a sender to label a sequence of packets as belonging to a specific "flow" (such as a continuous real-time video stream).
 - When intermediate routers see this label, they can forward all packets belonging to that flow along the exact same path with the exact same QoS treatment, without needing to deeply inspect transport-layer port numbers.

IPv6: Flow Label (Linux CLI)

Parsing in Linux CLI:

```
$ sudo tcpdump -vv -n ip6
```

Verbose Explanation of Command:

- **-vv:** Very verbose mode. Without the double 'v', tcpdump normally hides the flow label to save screen space.
- **-n:** Prevents DNS lookups so the terminal isn't overwhelmed with reverse-IP lag.
- **ip6:** Filters strictly for IPv6.
- **What to look for:** If a modern application (like a WebRTC video call) is actively utilizing flows, the output will distinctly display `flowlabel 0x5f3a1`. All subsequent packets in that specific video stream will share that exact same 20-bit hex identifier.

IPv6: Payload Length (Theory)

Book Reference: Kurose & Ross, Section 4.3.4

- **Bit Offsets:** Bits 32-47
- **Typical Values:** Varies based on application data size.
- **Verbose Explanation:**
 - This 16-bit field functions similarly to the IPv4 "Total Length" field, but with a critical distinction.
 - In IPv4, Total Length includes the header. In IPv6, the **Payload Length** strictly measures the number of bytes that follow the fixed 40-byte IPv6 header.
 - It includes any upper-layer data (TCP/UDP) as well as any IPv6 "Extension Headers" that might be chained between the standard IPv6 header and the transport layer.

IPv6: Payload Length (Linux CLI)

Parsing in Linux CLI:

```
$ ping6 -s 50 ::1 > /dev/null &  
$ sudo tcpdump -v -c 1 ip6
```

Verbose Explanation of Command:

- `ping6 -s 50 ::1`: We use the IPv6-specific ping command, targeting the IPv6 loopback address (`::1`). We force a payload of 50 bytes.
- `-v`: Verbose mode extracts the length.
- **What to look for:** The output will display `plen 58`.
- **The Math:** Why 58? 50 bytes of ping payload + 8 bytes of ICMPv6 header = 58 bytes. Notice it specifically ignores the 40-byte IPv6 header, proving that `plen` is strictly the payload size!

IPv6: Next Header (Theory)

Book Reference: Kurose & Ross, Section 4.3.4

- **Bit Offsets:** Bits 48-55
- **Typical Values:** 6 (TCP), 17 (UDP), 58 (ICMPv6), 43 (Routing Header).
- **Verbose Explanation:**
 - This 8-bit field completely replaces the IPv4 "Protocol" field.
 - Primarily, it tells the destination host to which transport-layer protocol the payload should be delivered (demultiplexing).
 - However, its secondary brilliance is chaining. If an IPv6 datagram requires options (like IPSec encryption or special routing), the Next Header value points to an "Extension Header." That extension header will then point to the next, forming a linked list until the final TCP/UDP payload is reached.

IPv6: Next Header (Linux CLI)

Parsing in Linux CLI:

```
$ sudo tshark -O ipv6 -c 1 -Y "icmpv6"
```

Verbose Explanation of Command:

- `-O ipv6`: Expands the IPv6 tree in terminal-wireshark.
- `-Y "icmpv6"`: Strictly filters the capture so we only see ICMPv6 traffic.
- **What to look for:** In the highly verbose output, tshark will explicitly parse the 48-55 bit offset and display: Next Header: ICMPv6 (58).
- If we had captured an SSH connection over IPv6, it would dynamically change to read Next Header: TCP (6).

IPv6: Hop Limit (Theory)

Book Reference: Kurose & Ross, Section 4.3.4

- **Bit Offsets:** Bits 56-63
- **Typical Values:** 64 (standard OS default).
- **Verbose Explanation:**
 - This 8-bit field is the exact functional replacement for the IPv4 Time-to-Live (TTL) field.
 - The designers renamed it "Hop Limit" because it was a more accurate description of what the field actually does: it limits the number of router hops, not the amount of physical "time" in seconds.
 - Just like IPv4, every IPv6 router must decrement this integer by 1. If it hits 0, the packet is discarded to kill routing loops.

IPv6: Hop Limit (Linux CLI)

Parsing in Linux CLI:

```
$ ping6 -c 1 ipv6.google.com
```

Verbose Explanation of Command:

- `ping6`: The IPv6 variant of the ping command. We use it to trigger a round-trip packet to a massive IPv6-enabled entity like Google.
- **What to look for:** Unlike `tcpdump`, standard Linux tools naturally parse this. The terminal will print a reply line containing the string `hlim=117` (or similar).
- By looking at `hlim=117`, we can mathematically deduce that the Google server likely started with a Hop Limit of 128, and the packet traversed exactly 11 routers ($128 - 117 = 11$) to reach our machine!

IPv6: Source & Destination Addresses (Theory)

Book Reference: Kurose & Ross, Section 4.3.4

- **Bit Offsets:** Source (Bits 64-191), Destination (Bits 192-319)
- **Typical Values:** Hexadecimal blocks separated by colons (e.g., 2001:db8::1).
- **Verbose Explanation:**
 - This is the most significant change in IPv6. The address size was quadrupled from 32 bits (in IPv4) to a massive 128 bits.
 - This creates an unfathomably large address space (3.4×10^{38} addresses), ensuring the Internet will theoretically never run out of IPs again.
 - The Destination Address remains the primary key used by routers to execute longest-prefix matching algorithms against their massive routing tables.

IPv6: Source & Destination Addresses (Linux CLI)

Parsing in Linux CLI:

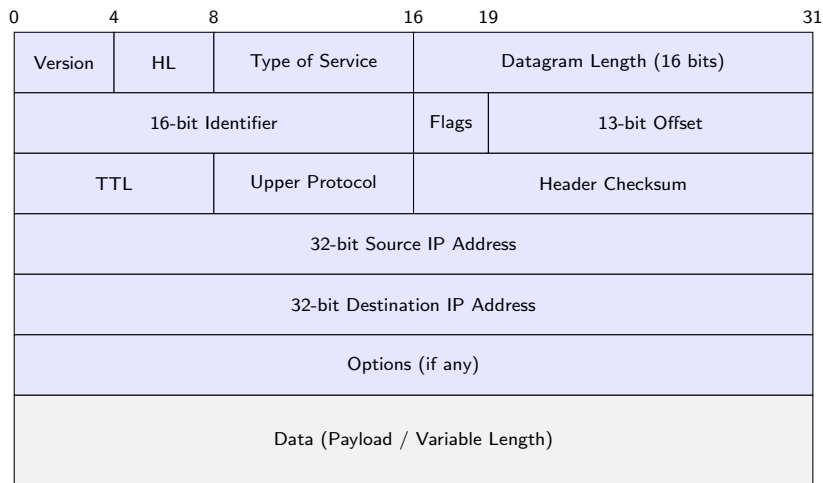
```
$ sudo tcpdump -n ip6
```

Verbose Explanation of Command:

- **-n:** This flag is absolutely mandatory when working with IPv6 in the CLI. Because IPv6 addresses are so long, reverse-DNS lookups take immense amounts of time and often fail. The **-n** flag forces instantaneous hexadecimal rendering.
- **What to look for:** The output will display giant hexadecimal strings representing the 128-bit routing logic, such as:
 - 2607:f8b0:400b:802::200e > 2001:4860:4860::8888.
- Notice how consecutive blocks of zeros are truncated using the double colon (::) shorthand defined by the IPv6 standard.

IPv4 Datagram Format (Schematic)

Book Reference: Kurose & Ross, Section 4.3.1



IPv6 Datagram Format (Schematic)

Book Reference: Kurose & Ross, Section 4.3.4

